PSP IA-1 — WORKSHEET SOLVED (BTech_PSP_IA1_Worksheet.pdf)

12 questions, full answers. Syllabus L1–L12. Every code answer below was executed and its output verified.

Source: the worksheet PDF you attached (2 pages: Q1–Q9 + 3-question 30-mark mock suite).

Section 1 — Flowcharts & Basic Syntax (L1–L3)

Q1. Flowchart: median of three numbers

Answer (steps to draw):

1. Start (oval).
2. Read a, b, c (three parallelograms or one "Read a, b, c").
3. Decision: `a > b`?
- Yes → swap a and b (process box: `t=a; a=b; b=t`).
4. Decision: `b > c`?
- Yes → swap b and c → then decision `a > b`? Yes → swap a and b.
5. Output **b** (parallelogram) — b is the median.
6. End (oval).

Shapes to remember: oval = start/end · parallelogram = input/output · rectangle = process · **diamond = decision** (two labelled exits).

Q2. Type casting & operators — predict output

```
a = 15 // 4          # 3        (int, floor division)

b = 15 % 4           # 3        (int, remainder)

c = float(a) + b     # 6.0      (float + int → float)

d = str(c) + "2.0"   # "6.02.0" (string concatenation)

print(f"a: {type(a)}, c: {type(c)}, d: {d}")
```

Output (verified):

```
a: <class 'int'>, c: <class 'float'>, d: 6.02.0
```

Why: `//` floors to int; `float(a)` makes c a float so `+b` stays float; `str(c)` turns 6.0 into `"6.0"` and `+ "2.0"` concatenates — no arithmetic happens in the last line.

Q3. Minutes → Hours and Remaining Minutes

```
m = int(input())

h, r = divmod(m, 60)

print(f"{h} Hours and {r} Minutes")
```

135 → `2 Hours and 15 Minutes`.

Section 2 — Conditionals & Logical Operators (L4–L6)

Q4. Triangle validator (nested if-else + ladder)

```
a = int(input()); b = int(input()); c = int(input())

if a + b > c and a + c > b and b + c > a:      # strict inequality

    if a == b and b == c:

        print("Equilateral")

    elif a == b or b == c or a == c:

        print("Isosceles")

    else:

        print("Scalene")

else:

    print("Not a triangle")
```

Note: the statement says *strictly greater* — use `>` (not `>=`), otherwise 1,2,3 wrongly "works".

Q5. Logical operator short-circuiting

```
x = 5

y = 10

if x > 10 and (y / 0 == 0):

    print("Path A")

elif x == 5 or (y / 0 == 0):

    print("Path B")

else:

    print("Path C")
```

Output: `Path B` — and no ZeroDivisionError.

Why: in `and`, Python evaluates left first; `x > 10` is False, so the right side `(y/0 == 0)` is **never evaluated** (short-circuit) → whole condition False. In `elif`, `x == 5` is True, so `or` short-circuits and `(y/0 == 0)` is again never run → condition True → Path B.

Section 3 — Modular Programming & Recursion (L7–L10)

Q6. `process_tax(income)` with conditional multiplier

```
def process_tax(income):

    if income <= 250000:

        rate = 0.0

    elif income <= 500000:

        rate = 0.05

    elif income <= 1000000:

        rate = 0.20

    else:

        rate = 0.30

    return round(income * rate, 2)


print(process_tax(600000))    # 120000.0
```

Scope point: `rate` is local to the function; nothing outside sees it. Return the value, do not print inside the function.

Q7. Recursive `sum_of_digits` + activation stack for n = 125

```
def sum_of_digits(n):

    if n < 10:                # base case: single digit

        return n

    return n % 10 + sum_of_digits(n // 10)


print(sum_of_digits(984))     # 21
```

Activation stack for n = 125:

```
sum_of_digits(125) → 125 % 10 = 5  + sum_of_digits(12)
  sum_of_digits(12) → 12 % 10 = 2  + sum_of_digits(1)
    sum_of_digits(1)  → base case → returns 1
  returns 2 + 1 = 3
returns 5 + 3 = 8
```

Result 8 (1+2+5). Frames unwind in reverse order.

Section 4 — Iterative Statements & Control Flow (L11–L12)

Q8. Convert the nested `for` to `while` (same output)

Original:

```
for i in range(1, 6):
    for j in range(i):
        print(i, end="")
    print()
```

While version (identical output — `1` then `22` then `333` …):

```
i = 1
while i <= 5:
    j = 0
    while j < i:
        print(i, end="")
        j += 1
    print()
    i += 1
```

Q9. Sentinel loop (-1.0 stops), count + running mean

```
count = 0

total = 0.0

while True:

    v = float(input())

    if v == -1.0:

        break

    total += v

    count += 1

print(count, total / count if count else 0.0)
```

Note: read as `float` (statement says floating-point); guard the mean when nothing was entered.

Section 5 — High-Fidelity Mock Exam Suite (30 marks)

Mock Q1 (5M) — Trace output

| v | Reason | Output line |

|---|---|---|

| 12 | even and divisible by 3 | `12: Divisible by 6` |

| 9 | not even, but divisible by 3 → or-branch, not even → Triple Only | `9: Triple Only` |

| 4 | even, not by 3 → Even Only | `4: Even Only` |

| 7 | neither → falls to final return | `7: Prime candidate` |

Exact output (verified):

```
12: Divisible by 6

9: Triple Only

4: Even Only

7: Prime candidate
```

Mock Q2 (10M) — `calculate_shipping(weight, distance, express_delivery)`

```
def calculate_shipping(weight, distance, express_delivery):

    if weight < 0 or distance < 0:

        return "Invalid Parameters"

    if weight < 5:

        rate = 50

    elif weight <= 20:

        rate = 80

    else:

        rate = 120

    total = rate * weight

    if distance > 500:

        total *= 1.15

    if express_delivery:

        total += 250

    return round(total, 2)
```

Checks: `(10, 600, True)` → 1170.0 · `(4, 100, False)` → 200.0 · `(-1, 10, False)` → `Invalid Parameters`.

Marking points: validation first (return string, no crash) · bracket boundaries (5 and 20 belong to the middle bracket as written "5kg to 20kg") · 1.15 on the weight total before the flat fee.

Mock Q3 (15M) — Alternating square series $S = 1^2 - 2^2 + 3^2 - 4^2 \pm N^2$

Alpha — for loop:

```
n = int(input())

total = 0

for i in range(1, n + 1):

    total += i * i if i % 2 else -(i * i)

print(total)
```

Beta — pure recursion (no external mutation):

```
def alt_sum(n):

    if n == 0:

        return 0

    term = n * n if n % 2 else -(n * n)

    return term + alt_sum(n - 1)


print(alt_sum(int(input())))
```

Both verified: n=4 → **−10** (1−4+9−16) · n=5 → **15**.

How to use this tonight

1. Do Q2, Q5, Mock Q1 **on paper** first (trace questions = guaranteed marks), check against above.
2. Then write Q3, Q4, Mock Q2, Mock Q3 Alpha from memory — these are the "write a program" marks.
3. Q7 stack trace: memorise the 125 example — same question shape reappears.
4. Q1 flowchart: memorise the median-of-three swap chain (3 decisions).